



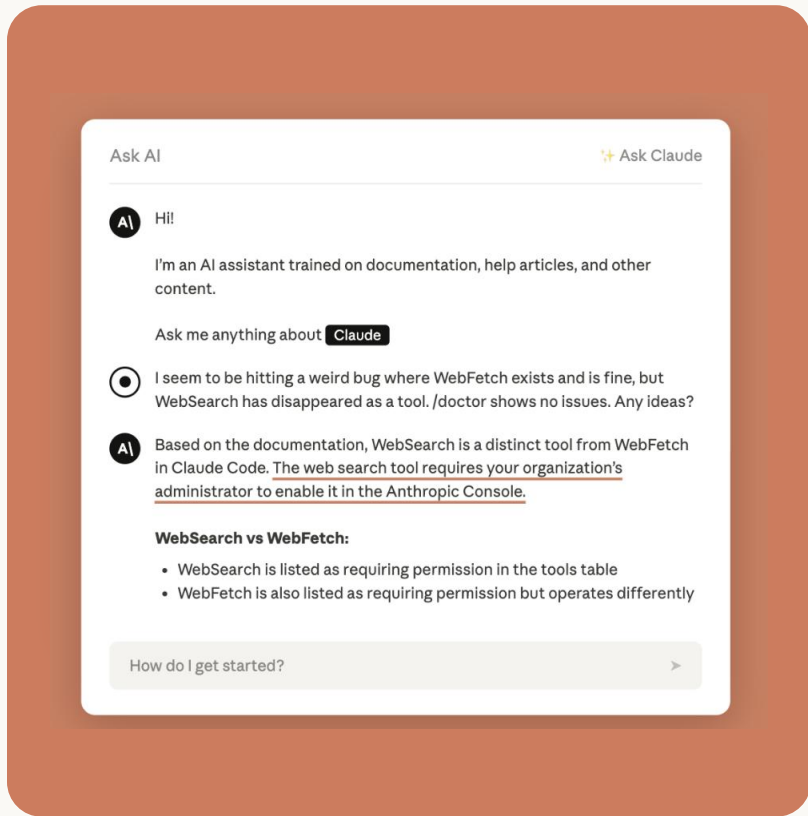
Claude Code Foundations

Designed for Claude Code Champions Hackathon

TIP 01

Ask Claude

If you get stuck, “Ask Claude” within code.claude.com/docs, or simply ask Claude Code directly.



AGENDA

What we'll cover

- | | | |
|----|-------------------------|---------|
| 01 | History of agents | 5 mins |
| 02 | Introducing Claude Code | 10 mins |
| 03 | Best practices | 15 mins |
| 04 | Live demo | 20 mins |
| 05 | Q&A and takeaways | 10 mins |

YOUR HOSTS

Who you're learning with



Alisa Feng

Customer Success • Beneficial
Deployments



Ado Kukic

Claude Code Hackathon Lead

 **Claude Code Docs**

code.claude.com/docs read setup guidance and Ask
Claude anytime.

 **Anthropic Academy**

<https://academy.claude.com/code> Asynchronous
courses covering Code, Cowork, AI Fluency, and
more.



PART ONE • 5 MINS

A history of agents

DEFINITION

What are agents?

Agents are AI systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.



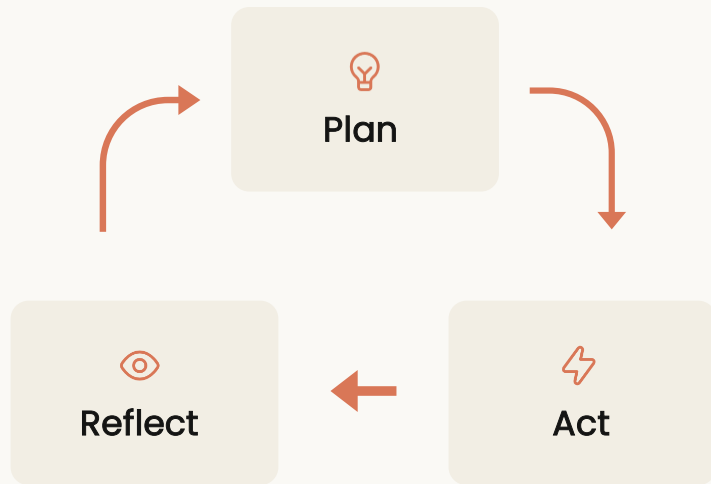
With a goal



Using tools

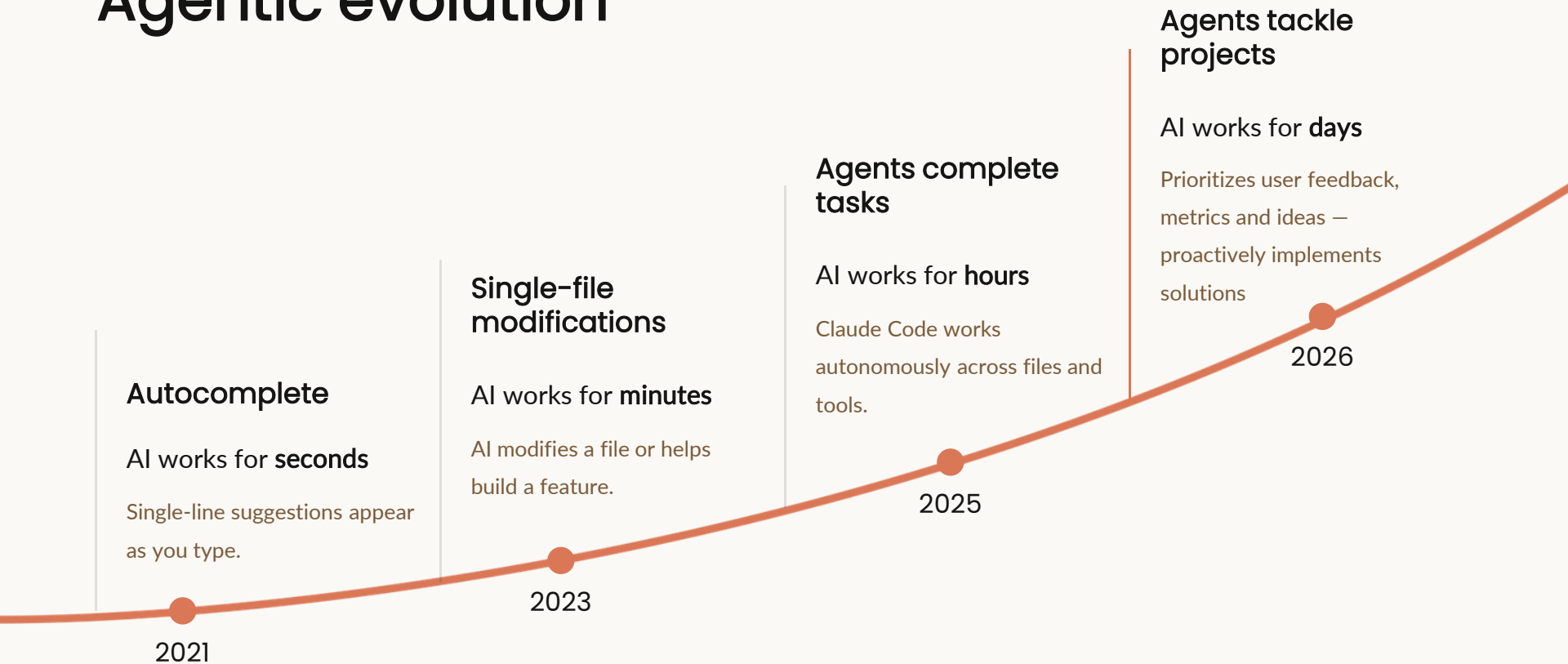


In a
loop



HOW WE GOT HERE

Agentic evolution



UNLOCKING VALUE FROM AGENTS

How Anthropic uses Claude Code

Product Development

Fast prototyping, test generation, bug fixes

Data Science

Dashboard apps, zero-dependency task delegation

Inference

Codebase comprehension, cross-language translation

Security Engineering

Infrastructure debugging, documentation & runbooks

Product Design

Rapid interactive prototyping, compliance copy

Legal

Workflow automation, team coordination tools

RL Engineering

Test generation, code review, K8s operations

Growth Marketing

Automated ad generation, prompt engineering

API

Workflow planning, model iteration testing



PART TWO · 10 MINS

Introducing Claude Code

Claude Code Foundations

SAME LLM, FOUR SURFACES

Four ways to use Claude



Claude Code

Delegate coding tasks

In your terminal, IDE, or
desktop app



Claude.ai

Chat with Claude

In your browser or the
desktop/mobile app



Cowork

Delegate knowledge
work

In the desktop/mobile app



Claude API

Integrated in your
application

In a Claude console org

GET SET UP

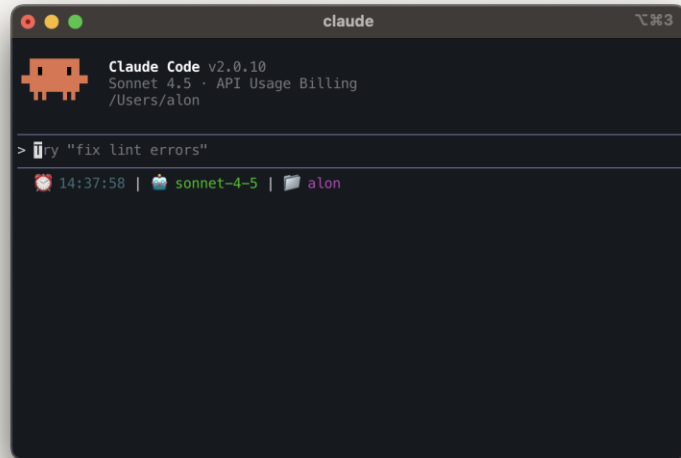
Install Claude Code

An agentic command-line tool that lets you delegate complex tasks using natural language, while maintaining transparency and control.

Run this in your terminal to install:

```
curl -fsSL https://claude.ai/install.sh | bash
```

[Installation guide](#) →



Claude Code runs wherever you work



Claude Code CLI

Bring Claude directly
into your terminal



Web + Desktop

Delegate tasks from
your phone or
browser



IDE extensions

Integrate with VS
Code and JetBrains



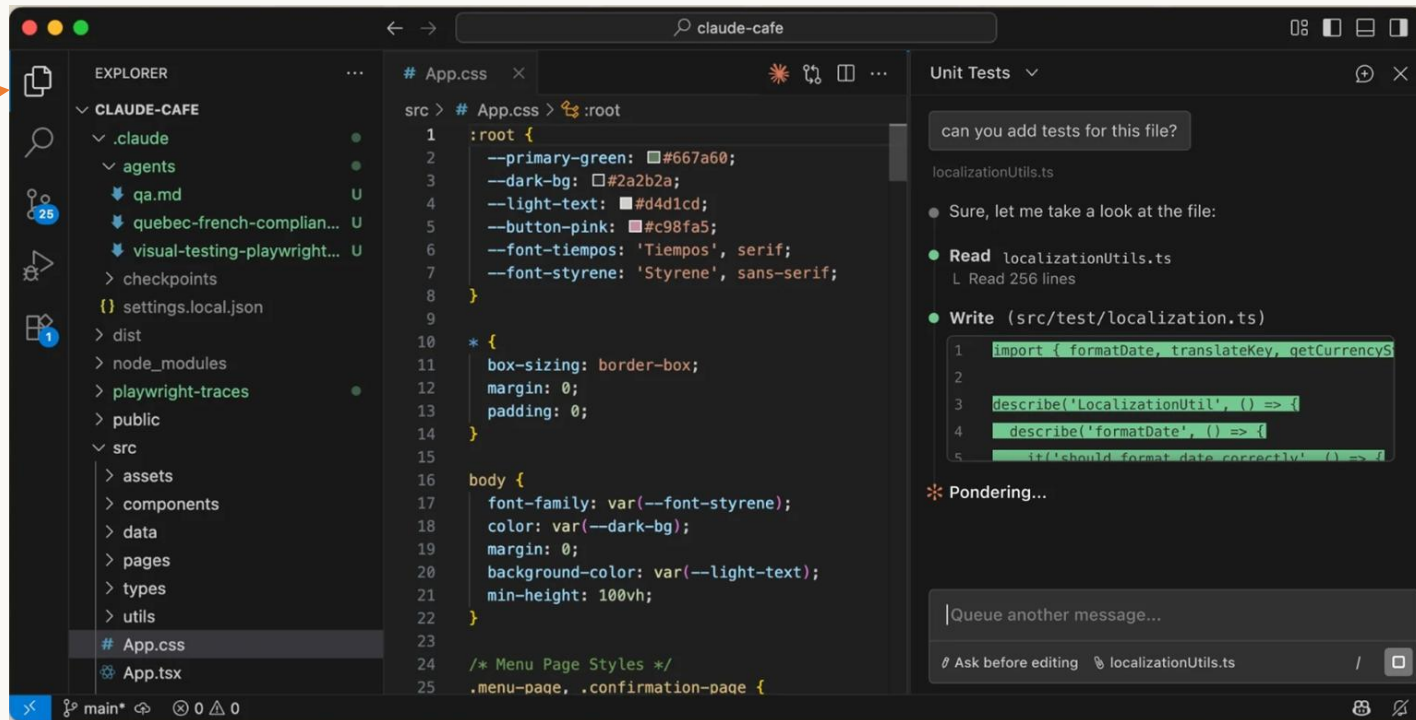
Claude Agent SDK

Work in Slack,
GitHub, mobile, and
more

IN AN IDE

Claude in VS Code

Explore
files





Claude Code augments your entire software development lifecycle



Discover



Design



Build



Deploy



Support

From exploring a new codebase to deploying and monitoring in production

Claude Code across your SDLC

1. Discover

Explore codebase and history

Search documentation

Onboard & Setup

2. Design

Plan project

Develop tech specs

Define architecture

3. Build

Implement code

Write and execute tests

Create commits and PRs

4. Deploy

Automate CI/CD

Configure environments

Manage deployments

5. Support & Scale

Debug errors

Large-scale refactor

Monitor usage & performance

Claude Code can use all of your team's CLI tools (e.g., git, docker, bq) so you can focus on solutions and strategy.



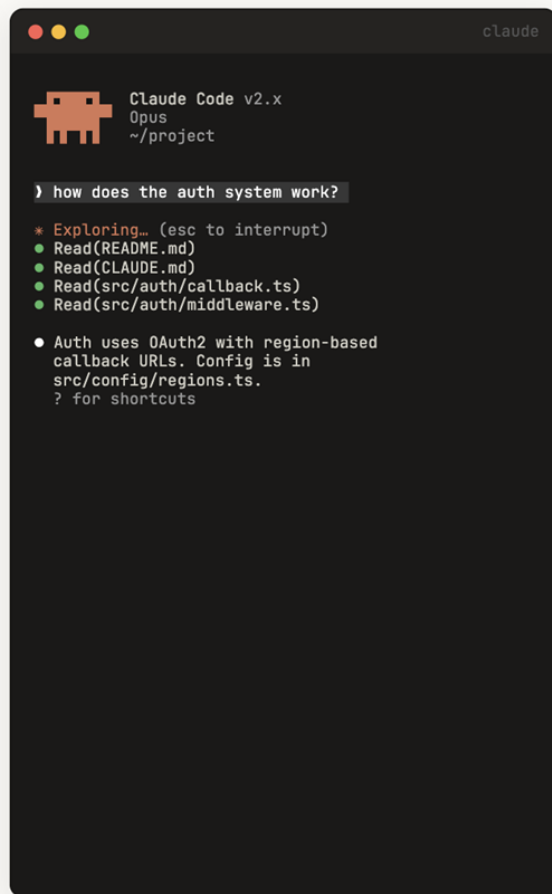
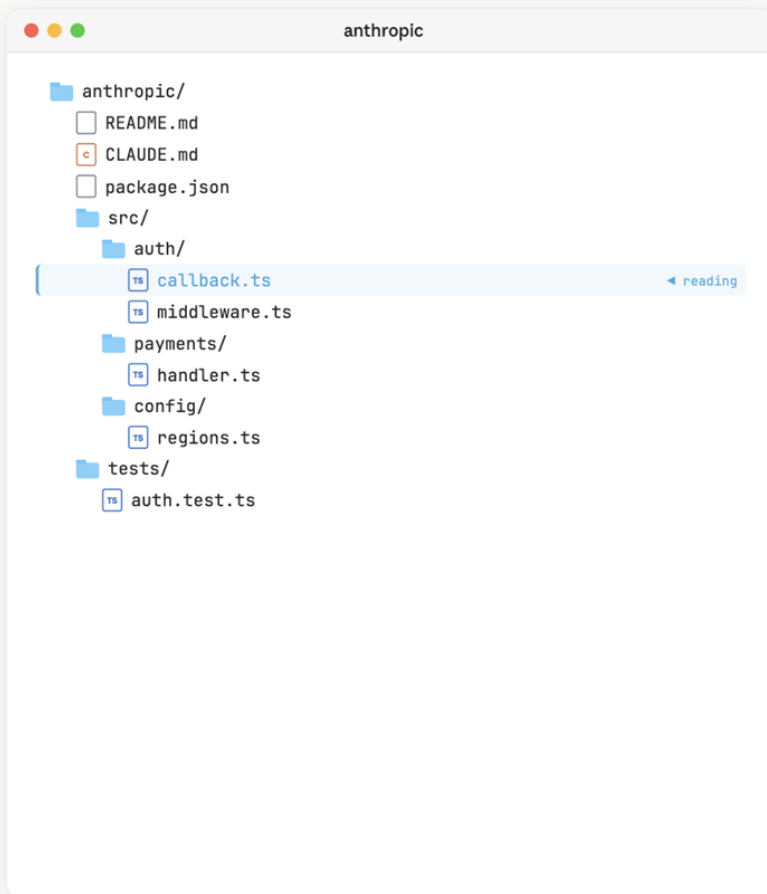
Discover

Design

Build

Deploy

Support





Discover



Design



Build



Deploy



Support

RFC-2025-003.md

RFC: OAuth2 Callback Refactor

Status: Draft · Author: Claude · Date: Feb 2025

Problem

EU users are redirected to US callback endpoint. The redirect_uri is hardcoded to AUTH_BASE_URL instead of using the region config.

Proposed Solution

- Replace AUTH_BASE_URL with config.baseUrl per region
- Add URL validation via isValidCallbackUrl()
- Add error handling for invalid redirects

API Changes

```
// Before
const redirectUri = AUTH_BASE_URL + "/callback";

// After
const redirectUri = config.baseUrl + "/callback";
if (!isValidCallbackUrl(redirectUri)) {
  throw new InvalidRedirectError(redirectUri);
}
```



claude

Claude Code v2.x

Opus

~/project

› write an RFC for fixing the callback

- Read(src/auth/callback.ts)
- Read(src/config/regions.ts)
- └ Write(docs/RFC-2025-003.md)
- Drafted RFC with problem statement, proposed solution, and rollout plan. ? for shortcuts

```
callback.ts

41 const config = getRegionConfig(user);
42 const callbackUrl = config.authEndpoint;
43
44 // Redirect to OAuth provider
45 -- const redirectUri = AUTH_BASE_URL + "/callback";
46 ++ const redirectUri = config.baseUrl + "/callback";
47 ++ if (!isValidCallbackUrl(redirectUri)) {
48 ++   throw new InvalidRedirectError(redirectUri);
49 ++ }
50 return oauth.authorize({ redirectUri });
```

claude

 Claude Code v2.x
Opus
~/project

- Read(src/auth/callback.ts)
 - | Write(src/auth/callback.ts)
 - + Use config.baseUrl instead of AUTH_BASE_URL
 - + Add URL validation
 - | Write(src/utils/validation.ts)

✓ Fixed 2 files, +47 -12 lines
? for shortcuts



anthropics / anthropic Private

Code Issues 345 **Pull requests** 5k+ Actions Security

 **Fix OAuth callback URL mismatch #847**

Open sarah-chen wants to merge 3 commits into `main` from `fix/oauth-callback`

Conversation 3 Commits 3 **Checks** 4 Files changed 3

✓ **Build & Compile** Successful in 12s

✓ **Tests (23/23)** Successful in 20s

✓ **Lint & Format** Successful in 28s

✓ **claude-review** Successful in 36s

✓ **All checks have passed**
4 successful checks

Merge pull request

 Claude Code v2.x
Opus
~/project

/pr-prep

- ✓ PR #847 created
- CI checks running...
- ✓ Build passed
- ✓ Tests passed (23/23)
- ✓ claude-review: approved
- ✓ Deployed to staging
- ? for shortcuts

Pro

Iss

Dis

Per

Ale

Rel

JS

Issues > TypeError

ERROR · JAVASCRIPT

TypeError: Cannot read properties of undefined (reading 'url')

847 events 312 users First seen 2h ago Last seen 3m ago

Details Activity User Feedback Tags

EVENTS



Exception

Most recent call first

processTicksAndRejections node:internal/process:82

Router.handle express/router:174

handleOAuthCallback src/auth/callback.ts line 47

```
44 // Redirect to OAuth provider
45 const config = getRegionConfig(user);
46 const callbackUrl = config.authEndpoint;
47 const redirectUri = AUTH_BASE_URL +
  "/callback";
48 return oauth.authorize({ redirectUri });
```

STATUS

Unresolved

PRIORITY

Critical

ASSIGNEE

Sarah C.

TAG SUMMARY

browser Chrome

region eu-we

release v2.1.

claude

 Claude Code v2.x
Opus
~/project

▲ TypeError at callback.ts:47 (847 events)

● Read(src/auth/callback.ts:47)

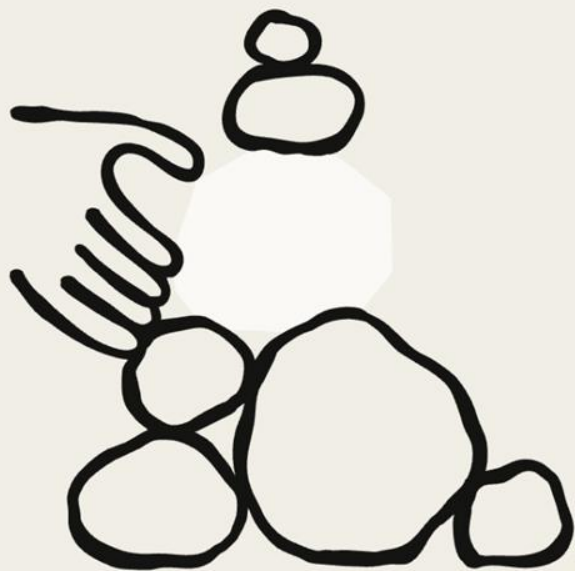
● Root cause: AUTH_BASE_URL hardcoded
This was fixed in PR #847

✓ Hotfix deployed
? for shortcuts



PART THREE · 15 MINS

Best practices



MODELS

Pick the right brain for the job

- **Opus** Most capable
Architecture, gnarly refactors, unfamiliar codebases. Slower, more expensive, worth it.
- **Sonnet** Balanced
Most edits, most debugging, most everything. The default for a reason.
- **Haiku** Fastest
Quick lookups, formatting, background subagent work. Cheap and fast.

Switch models inside Claude Code with `/model`. It lists what's available on your account — no need to memorize IDs.

Which Model Do I Use When?

	Opus 4.8	Sonnet 4.6	Haiku 4.5
Description	The world's best model for coding, enterprise agents, and professional work.	Approaches Opus levels of intelligence, enabling scalable, high-quality task completion, making it ideal for scaled use cases.	Most efficient model delivering near-frontier performance at accessible cost and speed.
Recommended for	The hardest, most complex work — problems that need peak intelligence and sustained autonomy.	High-volume workloads where you need near-Opus intelligence at scale.	Use cases where you need to balance performance with cost and speed.
Best for / when to use	• Long-horizon agent tasks • Deep, complex coding • Precision enterprise workflows (finance, legal) • Computer use automation • Cybersecurity	• High-volume production deployments • Agentic workflows and tool use • Fast iteration and prototyping • Cost-sensitive workloads + strong intelligence • Computer use automation • Cybersecurity	• Powering free tier user experiences • Latency-sensitive experiences • Executing tasks planned by more capable models • Efficient research
Cost	\$5.00 / \$25.00 per MTok	\$3.00 / \$15.00 per MTok	\$1.00 / \$5.00 per MTok
Context	1M	1M	200K

BEST PRACTICE

Plan Mode

Think before you act. Review the approach before any files change.

How to activate

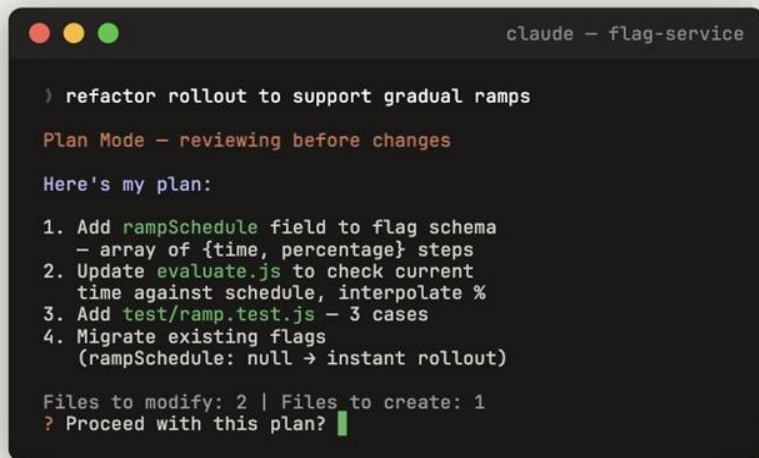
Press `shift+tab` to cycle modes — twice lands on Plan. Or type `/plan` before any prompt. Works mid-session: you can plan one task and execute the next.

When to use it

- Architectural changes
- Multi-file refactors
- Unfamiliar codebases
- Any task where mistakes are costly to undo

The value

You review the approach before any files are touched. Ask follow-ups, challenge assumptions, refine iteratively — then execute once, with confidence. The plan itself becomes documentation of *why*.

A terminal window titled 'claude - flag-service' with a dark background and light-colored text. It shows a sequence of commands and responses from Claude AI. The first command is 'refactor rollout to support gradual ramps'. The response is 'Plan Mode - reviewing before changes'. The user then says 'Here's my plan:'. The response is a numbered list of four steps: 1. Add 'rampSchedule' field to flag schema (array of {time, percentage} steps), 2. Update 'evaluate.js' to check current time against schedule, interpolate %, 3. Add 'test/ramp.test.js' - 3 cases, 4. Migrate existing flags (rampSchedule: null → instant rollout). Below the list, it says 'Files to modify: 2 | Files to create: 1'. The final line is a prompt '? Proceed with this plan?' followed by a green cursor bar.

```
claude - flag-service

) refactor rollout to support gradual ramps

Plan Mode - reviewing before changes

Here's my plan:

1. Add rampSchedule field to flag schema
  - array of {time, percentage} steps
2. Update evaluate.js to check current
  time against schedule, interpolate %
3. Add test/ramp.test.js - 3 cases
4. Migrate existing flags
  (rampSchedule: null → instant rollout)

Files to modify: 2 | Files to create: 1
? Proceed with this plan? █
```

LIVE DEMO 1

Have Claude plan a project

Prompt A

I'm a volunteer coordinator at Mason Public Library. I'm responsible for scheduling volunteers for the months of June and July. I want you to create a portal that schedules 3 shifts per day, M-Sunday, 9am-12pm, 12-3pm, 3-6pm. Here's a CSV of my available volunteers and their preferred availability. Limit each volunteer to 2 shifts per week. **Without building anything, tell me your plan. Ask me about any additional context you need.**

Claude Code is as good as the
context you give it

What keeps Claude running smoothly

Automatic features

Auto-memory

Context accumulates automatically during your session. Run `/memory` to manage it.

Auto-compaction

Claude periodically summarizes your context.

Proactive steps

CLAUDE.md

An onboarding doc for your project.

@-mentions

Provide direct file paths to save Claude time.

.ignore files

Ignore irrelevant files.

Claude Code toolkit

Skills

Codified workflows Claude can reuse.

MCPs

Connectors to other systems.

Slash commands

`/context`

`/compact`

`/clear`

`/resume`

PROACTIVE CONTEXT MANAGEMENT

CLAUDE.md

Similar to a README, but for Claude. It holds instructions on project structure, common commands, personal styling tips, and more.



Tips

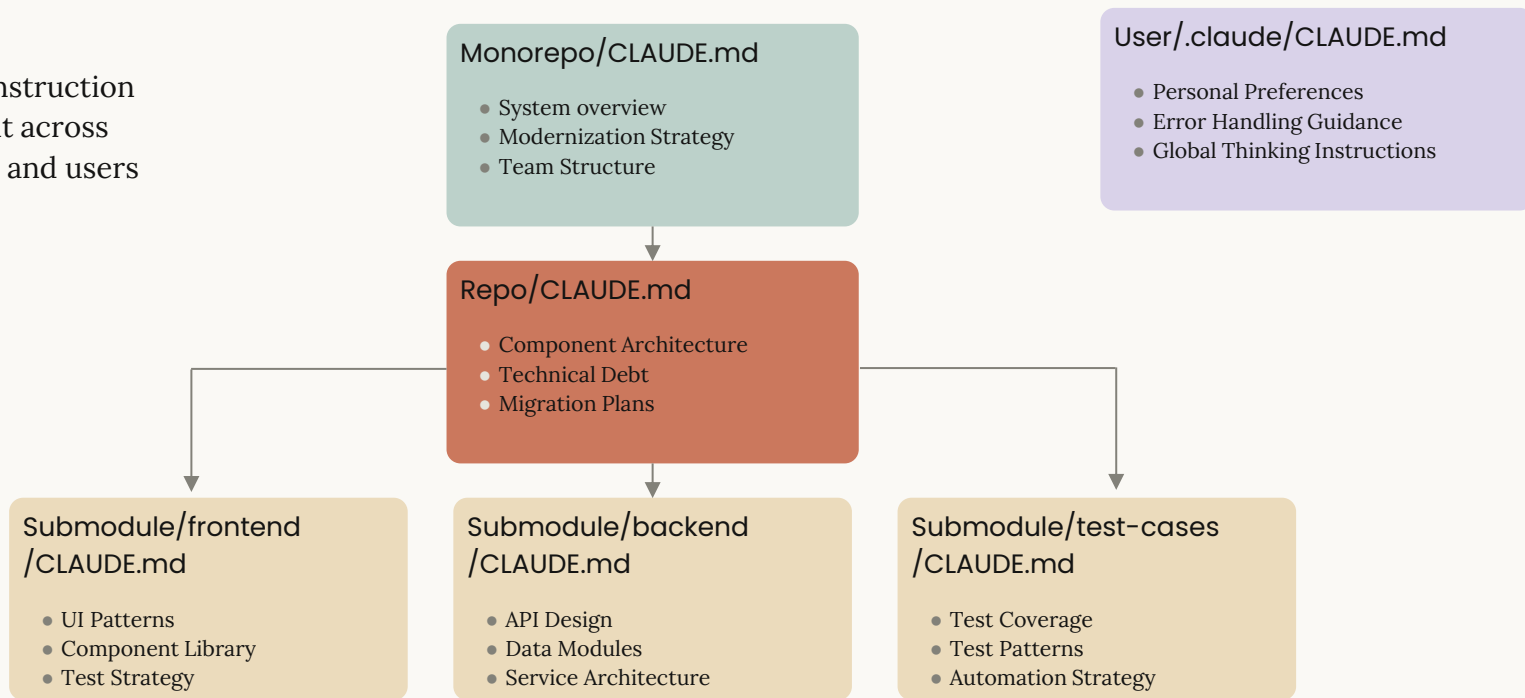
Use `/init` to draft a CLAUDE.md from your repo anytime. Include things not present in the code itself and aim for under 200 lines.

CLAUDE.md

```
1 # flag-service
2
3 ## What the code can't tell you
4 Hash bucketing uses `% 100`, not `% 101`. The off-by-one
5 means a "100%" rollout misses ~1% of users. It's a bug.
6
7 ## Constraints
8 - NEVER auto-enable a flag in prod. Use `/rollout`.
9   We took down checkout for 40min in Jan. The skill has guards.
10 - Flags older than 90 days with 100% rollout: delete the flag,
11   inline the code path. Don't let them accumulate.
12
13 ## Who to ask
14 Rollout policy: #platform-eng. Flag store schema: @priya.s
```

Hierarchical CLAUDE.md

Updatable instruction
management across
repositories and users



PROACTIVE CONTEXT MANAGEMENT

@-file mentions

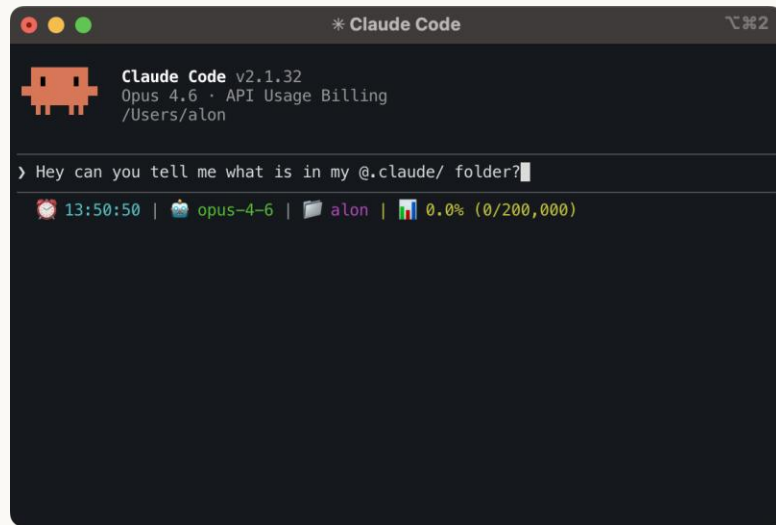
@-mentioned file contents are automatically provided to Claude at message send time.

Alternatively, “Hey can you tell me how Foo.tsx works” means the agent has to call the Read(Foo.tsx) tool itself.



Tip

@ a specific file in your CLAUDE.md to have Claude reference it at the start of each session.



Tools to help manage context

Automatic

Auto-memory

Context accumulates as you work on a project.

Auto-compaction

Claude periodically summarizes your context.

Proactive setup

CLAUDE.md

Claude's README file.

@-mentions

Provide direct file paths.

.ignore files

Ignore irrelevant files.

Claude Code toolkit

Skills

Progressively disclose the context you need.

MCPs

"Connectors" between Claude and other systems.

Slash commands

/context

Visualize your context.

/compact

Condense context anytime.

/clear

Clear context for a new task.

Try these out in the upcoming build-along

POWER FEATURE

Anatomy of a Skill

Domain expertise, codified — not a prompt you keep retyping.

What is a Skill?

A structured instruction set that teaches Claude a workflow. Carries context, decision logic, and multi-step processes.

Reusable

Shareable

Composable

When to create one?

- You've typed the same prompt three times
- Tribal knowledge lives in one person's head
- Consistency matters more than flexibility
- A workflow has compliance steps that can't be skipped

A tool is a screwdriver. A Skill is an electrician's training.

Skills can read `${CLAUDE_EFFORT}` — branch behavior on how hard Claude is thinking.

Lock it down? `disallowed-tools: [Bash, WebFetch]` removes those tools while the skill runs — the workflow *is* the guardrail.

Edited SKILL.md? `/reload-skills` re-scans without restarting the session.

`.claude/skills/promote-flag/SKILL.md`

```
1  ---
2  name: promote-flag
3  description: Promote a flag from staging to prod
4  ---
5
6  ## Preconditions
7  - Staging at target % for 48h minimum
8  - No open incidents tagged with this flag
9
10 ## Steps
11 1. Query staging metrics via flag-store MCP
12 2. Check incident API for the flag key
13 3. Diff staging % vs prod %, show the gap
14 4. Bump prod by same delta. Snapshot first.
15 5. Emit rollback command in the summary
16
17 ## Constraints
18 - Never exceed 50% in one step
19 - Prod changes require @mention confirmation
20
21 ## References
22 - lib/flags.js
23 - docs/rollout-policy.md
```

POWER FEATURE

MCPs (“Connectors”)

“Model context protocol” is a way for Claude to read and write to other tools.

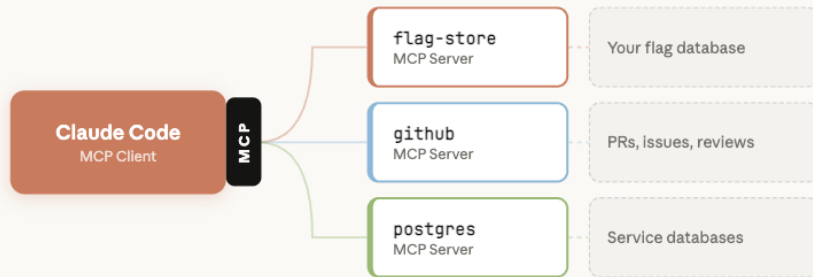
Skills don't have to involve MCPs, but they often do. Use this instead of adding all of your context manually.

 Google Drive

 Slack

 GitHub

 Salesforce



SLASH COMMANDS

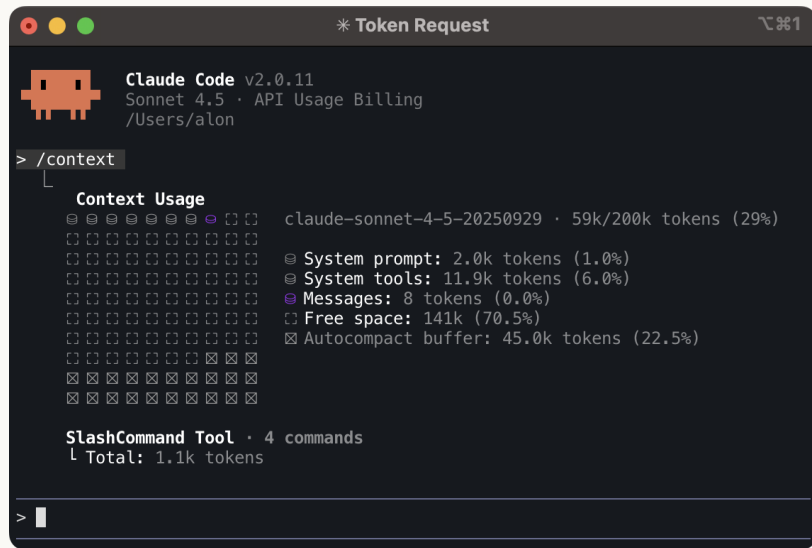
Manage context on demand

/context Visualize what's in your context window, including available MCPs.

/compact Summarize current context to a smaller size before Claude auto-compacts.

/clear Start with a clean slate when shifting to a new task.

Note: some models have a 1M context window, which requires less compaction.



The screenshot shows the Claude Code v2.0.11 interface. The title bar indicates a "Token Request" window. The main content area displays the output of the `/context` slash command. It shows a visual representation of the context window with a grid of icons. The output includes the following information:

- Context Usage:** A visual representation of the context window showing the usage of various components.
- System prompt:** 2.0k tokens (1.0%)
- System tools:** 11.9k tokens (6.0%)
- Messages:** 8 tokens (0.0%)
- Free space:** 141k (70.5%)
- Autocompact buffer:** 45.0k tokens (22.5%)

At the bottom, it shows the "SlashCommand Tool" with 4 commands and a total of 1.1k tokens.

Tips for managing Claude-versations

/btw Chat without interrupting

Ask Claude something without interrupting its work — and use screenshots to “show, not tell.”

/rewind Jump back

Undo typos or experiment with different approaches in the conversation and code.

/resume Pick up where you left off

Continue from a previous Claude Code session.

Esc Interrupt the agent

Give Claude Code more context or steer in a new direction.

PART FOUR · 20 MINS · LIVE DEMO

Build a volunteer scheduling portal

Try some of the new commands

LIVE DEMO 2

Run it yourself

1. Open the terminal and type **claude** to start your session
2. Run **/model** to select Sonnet 4.6
3. Run **/mcp** to see your available connectors
4. Hit **Shift + Tab** to select plan mode
5. Give Claude prompt A → review the plan, answer questions, approve
6. Once built, give Claude prompt B (a screenshot can help debugging)
7. Run **/frontend-design**, **/rewind**, **/btw**, **/context**, and **/compact** as needed
8. Run **/init** to draft your CLAUDE.md
9. Ask Claude to turn the scheduling process into a Skill

Prompt A

I'm a volunteer coordinator at Mason Public Library. I'm responsible for scheduling volunteers for the month of June and July. I want you to create a portal that schedules 3 shifts per day, M-Sunday, 9am-12pm, 12-3pm, 3-6pm. Here's a CSV of my available volunteers and their preferred availability. Limit each volunteer to 2 shifts per week. Without building anything, tell me your plan. Ask me about any additional context you need.

Prompt B

Lavender Brown can't make any Saturday shifts this month. (1) Remove her from all Saturday assignments. (2) Backfill her spots using available volunteers still under their 8-hour limit. (3) Add a highlighted 'Open shifts – volunteers needed' section for anything you can't fill. Update the branding to match the font and colors from this website: <https://www.masonpl.org/>

Shortcuts worth muscle memory

Keystrokes and prompt-box prefixes you'll reach for every session.

! Bash mode

Run a shell command directly. Output lands in the conversation — no copy-paste.

```
!npm test !git log --oneline -5
```

@ Reference a file or agent

Pull a file into the prompt — or invoke a named subagent. Same key, typeahead picks.

```
why is @lib/evaluate.js excluding user...
```

^V Paste a screenshot

Drag or paste an image. Claude sees it. Good for error dialogs, mockups, diagrams.

```
[screenshot] make the header match this
```

^B Background a task

Long-running command? ctrl+b sends it to the background. Keep working, get notified when done.

```
npm run build [ctrl+b] → continues in ...
```

esc² Rewind the conversation

esc esc jumps back to an earlier turn. Undo a direction that went sideways without /clear.

```
[esc][esc] → pick a prior message to r...
```

⇧⇨ Toggle plan mode

Shift+Tab cycles auto → plan → auto. Plan mode reads everything, proposes changes, edits nothing.

```
[shift+tab] → plan indicator appears →...
```

^O / Search the transcript

ctrl+o opens transcript view. / to search, n/N to step, { } to jump between your prompts.

```
[ctrl+o] /useEffect [n] · { } jump pro...
```

^X^E External editor

Open the prompt in \$EDITOR. Readline-standard — write a long prompt in vim, save, it sends.

```
[ctrl+x ctrl+e] → vim → :wq
```

v / V Vim visual mode

Select text in the prompt with vim motions. Yank, delete, or change on a selection — full visual + visual-line.

```
V j j d → delete three lines
```

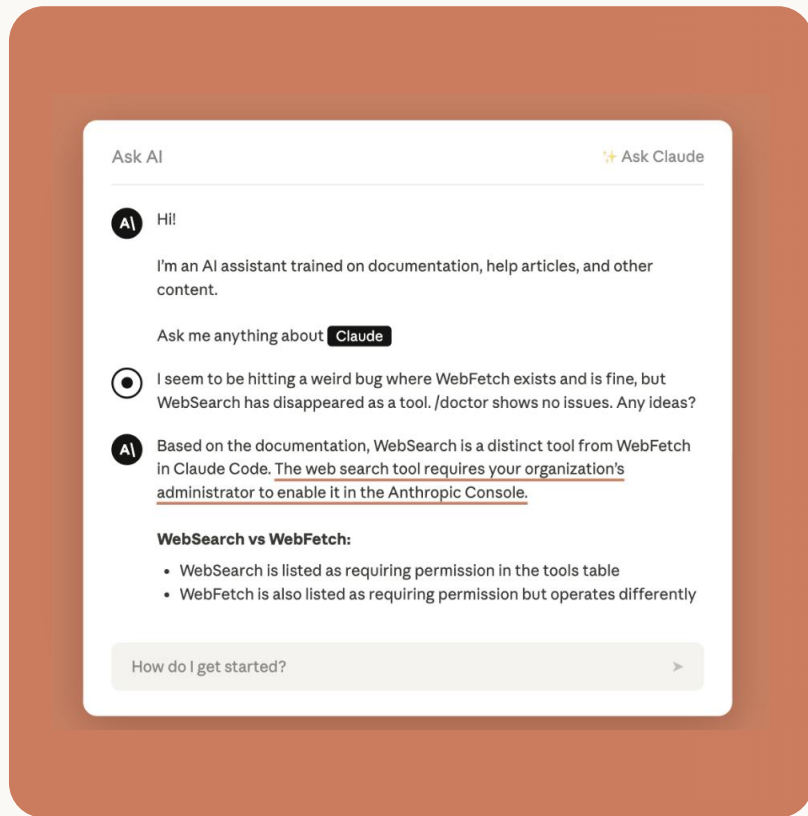
Bonus: ^R searches prompt history across all projects · hold space to dictate (voice push-to-talk, 20 languages). All remappable — ~/.claude/keybindings.json.

REMEMBER

Ask Claude

If you get stuck, “Ask Claude” within code.claude.com/docs, for simply ask Claude Code directly.

Want developer updates? Sign up for the [Anthropic Developer Newsletter](#)



First-time user checklist



Install and launch Claude

Launch Claude directly from your terminal, desktop app, or IDE.



Use Shift + Tab to run plan mode

Have Claude propose a plan for fixing a bug or building a dashboard.



Run `/context` and `/compact` if needed

Keep an eye on context and token use for a productive, cost-effective session.



Run `/init` to start a `CLAUDE.md` file

Review your `CLAUDE.md` and add 3-4 things Claude couldn't figure out.

KEEP LEARNING

Resources



Claude Code Quickstart

code.claude.com/docs/en/quickstart ↗



Claude Code CLI reference

code.claude.com/docs/en/cli-reference ↗



Anthropic Skills Repository

github.com/anthropics/skills ↗



Claude Cowork Quickstart

support.claude.com/en/articles/1334-5190-getting-started-with-cowork ↗



How Anthropic uses Claude Code

claude.com/blog/how-anthropic-teams-use-claude-code ↗



Connectors Directory

claude.com/connectors#connectors ↗



Questions?